

Point Cloud Ground Segmentation on GPU

Kuartis

Computer Vision, Machine Learning and Robotics

Calibration, Localization and Mapping Team

The main goal of this assignment is to create a 3D point cloud processing algorithm to be used in ground plane segmentation. For this assignment, you will process 3D point cloud data created by a LiDAR sensor onboard. A 3D point cloud consists of an array of points that each consist of X,Y,Z coordinates. The assignment should be run on a GPU using CUDA. You should also prepare a technical report for your implementation.

For further questions, you can contact Arda Efe Okay (efe.okay@kuartis.com). After the assignment, there will be another meeting session that will focus on the assignment results, implementation and report.

Details of the Assignment

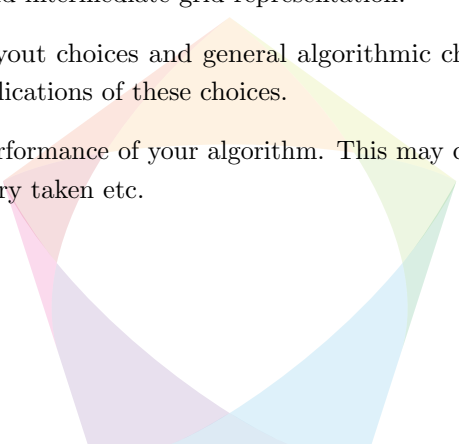
- The goal of the assignment is to segment points into ground/nonground points in the GPU. At the end, you should decide whether a given point in the point cloud data is ground or not.
- Sample point cloud data will be provided to you in PCD(.pcd) format. You can utilize Point Cloud Library (PCL) to read this data into array of 3D points. Also, all of the required parameters will be provided in a .yaml file. Next, the points will be transferred to GPU and the main algorithm will be run.
- **The Main Algorithm (on the GPU)**
 - First, you need to filter the data using the provided x and y limits. Also note that x axis represent the axis to the right, y axis represent the axis to the front and z axis represent the ground up.
 - You should rasterize the 3D point cloud data onto a fixed 2D grid. This is done by projecting the 3D points onto the 2D plane and determining which grid cell contains each point. Also note that all grids should lie on the $Z=0$ plane.
 - Using the points inside a grid, several values on the grid must be calculated. These are the number of points in the grid, the mean and variance of Z, and the minimum and maximum Z values. For efficient mean and variance calculation, you may check the Welford's algorithm. If a grid has less points than a threshold, it can be considered empty.
 - Next, by exploiting the Z variance values inside a grid, you must decide whether a grid is a ground or nonground grid. This classification is calculated using 3x3 patches. The cell in the center of the patch is considered ground if the variance of the cell is below a threshold. If the center patch is empty, the average variance of the patch may be substituted instead.
 - Last part is to actually classify the original 3D points. If a point is on a nonground grid, it is always nonground. If it falls into a ground grid, then its Z value is compared

with the grid's average Z value. If this difference is smaller than a threshold, it is classified as a ground point. You can save ground and non-ground point clouds separately in PCD(.pcd) format.

- You need to prepare both CPU and GPU implementations for the ground segmentation algorithm. You also need to add CMakeLists.txt to compile the code with gcc compiler. CMakeList should both compile C++ and CUDA modules.
- Please use PCL Library version 1.10.0
- You may add the other major library versions to the report: GCC and CUDA versions, etc.

Tips and Recommendations

- You can visualize 3D data using Point Cloud Library or Open3D. Provide visualizations for the input, output and intermediate grid representation.
- Summarize the data layout choices and general algorithmic choices you made in your report. Discuss the implications of these choices.
- Measure the runtime performance of your algorithm. This may consist of amount of time taken, amount of memory taken etc.



KU[^]ARTIS